

LAPORAN TUGAS BESAR
MILESTONE I – LEXICAL ANALYSIS
IF2224 TEORI BAHASA FORMAL DAN AUTOMATA
ARION INTERPRETER



Disusun oleh:

Kelompok 11

“noooRegex” - MMD

Kloce Paul William Saragih	13524040
Suryani Mulia Utami	13524042
Josh Reinhart Zidik	13524048
Syaqina Octavia Rizha	13524088

SEKOLAH TEKNIK ELEKTRO DAN INFORMATIKA
PROGRAM STUDI TEKNIK INFORMATIKA
INSTITUT TEKNOLOGI BANDUNG

2025/2026

DAFTAR ISI

BAB 1	1
PENDAHULUAN	1
BAB 2	2
DASAR TEORI	2
2.1. Deterministic Finite Automata (DFA)	2
2.2. Lexical Analysis	2
2.3. Lexer	2
2.4. Token	3
BAB 3	4
PERANCANGAN	4
3.1. Desain Diagram Transisi DFA	4
3.2. Teknologi.....	6
3.3. Struktur Program	7
3.4. Fungsi/Kelas Utama	7
3.5. Alur Kerja Program.....	10
3.6. Pendekatan Implementasi.....	11
BAB 4	13
HASIL IMPLEMENTASI	13
BAB 5	17
EKSPERIMEN.....	17
BAB 6	20
KESIMPULAN DAN SARAN	20
6.1. Kesimpulan	20
6.2. Saran	20
LAMPIRAN	21
REFERENSI	21

BAB 1

PENDAHULUAN

Bahasa pemrograman digunakan untuk menuliskan instruksi agar komputer dapat menjalankan suatu program tanpa harus menggunakan bahasa mesin secara langsung. Komputer hanya memahami bahasa mesin dalam bentuk biner (0 dan 1), sehingga bahasa pemrograman dikembangkan agar lebih mudah dipahami manusia. Secara umum, bahasa pemrograman dibedakan menjadi low-level yang dekat dengan bahasa mesin dan high-level yang lebih mendekati bahasa manusia. Semakin tinggi tingkat bahasanya, semakin banyak proses yang diperlukan untuk mengubahnya menjadi kode mesin.

Salah satu cara untuk menerjemahkan bahasa pemrograman ke dalam kode mesin adalah dengan menggunakan interpreter. Interpreter bekerja dengan membaca, menerjemahkan, dan mengeksekusi kode sumber secara baris demi baris. Pendekatan ini memudahkan dalam menemukan kesalahan karena error dapat langsung diketahui pada baris tertentu. Berbeda dengan compiler yang menerjemahkan seluruh source code menjadi kode mesin (misalnya dalam bentuk object file) sebelum program dijalankan.

Meskipun berbeda dalam cara eksekusi, keduanya tetap melalui tahapan analisis yang serupa. Proses dimulai dari Lexical Analysis yang memecah kode sumber menjadi token-token kecil seperti kata kunci, variabel, dan operator. Selanjutnya, Syntax Analysis yang memeriksa apakah susunan token tersebut sudah sesuai dengan aturan tata bahasa (*grammar*) yang berlaku. Setelah itu, Semantic Analysis memastikan bahwa makna dari kode tersebut benar. Tahap terakhir, Intermediate Code Generation yang mengubah kode ke bentuk perantara yang lebih mudah diproses sebelum akhirnya dieksekusi atau diterjemahkan lebih lanjut menjadi kode mesin.



Gambar 1 - Skema Kerja Interpreter

Sumber: GeeksForGeeks

BAB 2

DASAR TEORI

2.1. Deterministic Finite Automata (DFA)

Deterministic Finite Automaton (DFA) merupakan model komputasi yang digunakan untuk mengenali pola dalam suatu rangkaian simbol. DFA banyak digunakan dalam teori bahasa formal dan menjadi dasar dalam proses lexical analysis pada compiler maupun interpreter.

Secara formal, DFA didefinisikan sebagai suatu 5-tuple:

$$A = (Q, \Sigma, \delta, q_0, F)$$

- Q adalah himpunan hingga dari state
- Σ adalah himpunan simbol input (alphabet)
- δ adalah fungsi transisi yang memetakan pasangan state dan simbol input ke state berikutnya
- q_0 adalah state awal (start state)
- F adalah himpunan state akhir (final/accepting states)

Dalam proses kerjanya, DFA membaca input satu karakter pada satu waktu dan berpindah dari satu state ke state lain sesuai dengan fungsi transisi δ . Suatu string dikatakan diterima oleh DFA apabila proses pembacaan dimulai dari state awal dan berakhir pada salah satu state akhir setelah seluruh input dibaca.

Dalam konteks lexical analysis, DFA digunakan untuk mengenali pola-pola token seperti identifier, konstanta, maupun keyword. Setiap jenis token dapat direpresentasikan sebagai sebuah automata, sehingga proses pengenalan token dapat dilakukan secara sistematis melalui perpindahan state berdasarkan karakter yang dibaca. Dengan demikian, DFA menjadi dasar dalam implementasi lexer untuk mengelompokkan karakter menjadi token yang valid.

2.2. Lexical Analysis

Lexical Analysis merupakan tahap awal dalam proses kompilasi maupun interpretasi program. Pada tahap ini, kode sumber (*source code*) dibaca karakter demi karakter dari kiri ke kanan, kemudian dikelompokkan menjadi unit-unit terkecil yang bermakna, yang disebut **token**. Hasil dari proses ini adalah rangkaian token yang lebih terstruktur, sehingga tahap selanjutnya tidak lagi memproses karakter mentah, melainkan token yang sudah memiliki arti dalam konteks bahasa pemrograman. Proses ini dilakukan oleh komponen yang disebut **lexer**.

Proses pengenalan dan pembentukan token pada tahap ini umumnya dilakukan menggunakan konsep **Deterministic Finite Automaton (DFA)**. DFA digunakan untuk mengenali pola tertentu dari rangkaian karakter, misalnya membedakan apakah suatu string merupakan identifier, angka, atau keyword. Dengan menggunakan DFA, proses lexical analysis dapat dilakukan secara sistematis dan efisien karena setiap karakter yang dibaca akan menentukan transisi ke state berikutnya hingga terbentuk sebuah token yang valid.

2.3. Lexer

Lexer (lexical analyzer) merupakan komponen yang bertugas mengubah kode sumber menjadi rangkaian token. Lexer bekerja dengan membaca input karakter demi karakter, kemudian mencocokkannya dengan pola tertentu untuk menentukan jenis token yang sesuai. Dalam prosesnya, lexer tidak hanya menghasilkan jenis token, tetapi juga mengambil bagian teks asli dari source code yang disebut lexeme. Lexeme adalah representasi konkret dari token yang dikenali oleh lexer, yaitu urutan karakter dalam kode sumber yang sesuai dengan suatu pola token tertentu.

2.4. Token

Seperti yang telah dijelaskan sebelumnya, token adalah unit terkecil yang memiliki makna dalam program dan terbentuk dari sekumpulan karakter. Token menjadi dasar dalam pendefinisian grammar, karena tahap selanjutnya tidak lagi memproses karakter per karakter, melainkan kumpulan token yang sudah terklasifikasi.

Setiap token umumnya memiliki dua komponen, yaitu jenis (*type*) dan nilai (*value*). Jenis token menunjukkan kategori dari token tersebut, sedangkan nilai token merupakan isi atau representasi sebenarnya dari token. Beberapa kategori token di antaranya:

- **Keyword**, yaitu kata kunci yang memiliki makna khusus dalam bahasa (misalnya `'program'`, `'var'`, `'begin'`)
- **Identifier (ident)**, yaitu nama yang digunakan untuk variabel, fungsi, atau entitas lain
- **Konstanta**, seperti integer, real, karakter, dan string
- **Operator**, baik aritmatika, relasional, maupun logika
- **Delimiter (separator)**, yaitu simbol yang digunakan untuk memisahkan atau membatasi struktur dalam program, seperti `';`, `' '`, `'(, )'`, `'[, ]'`, `'::'`, dan `'.'`.

BAB 3

PERANCANGAN

3.1. Desain Diagram Transisi DFA

Berdasarkan diagram transisi DFA yang digunakan, automata ini terdiri dari **121 state** dengan **52 final state** yang merepresentasikan berbagai jenis token dalam bahasa pemrograman.

DFA didefinisikan sebagai berikut:

$$A = (Q, \Sigma, \delta, q_0, F)$$

- **Q (Himpunan State)**

Merupakan himpunan hingga dari seluruh state dalam automata. Pada implementasi ini, Q terdiri dari 121 state yang mencakup state awal, state antara, state akhir, serta dead state.

- **Σ (Alphabet)**

Merupakan himpunan simbol input yang dapat dibaca oleh DFA. Pada diagram, simbol ini meliputi huruf (a–z, A–Z), digit (0–9), serta simbol khusus seperti +, -, *, /, (,), :, dan lain-lain.

- **δ (Fungsi Transisi)**

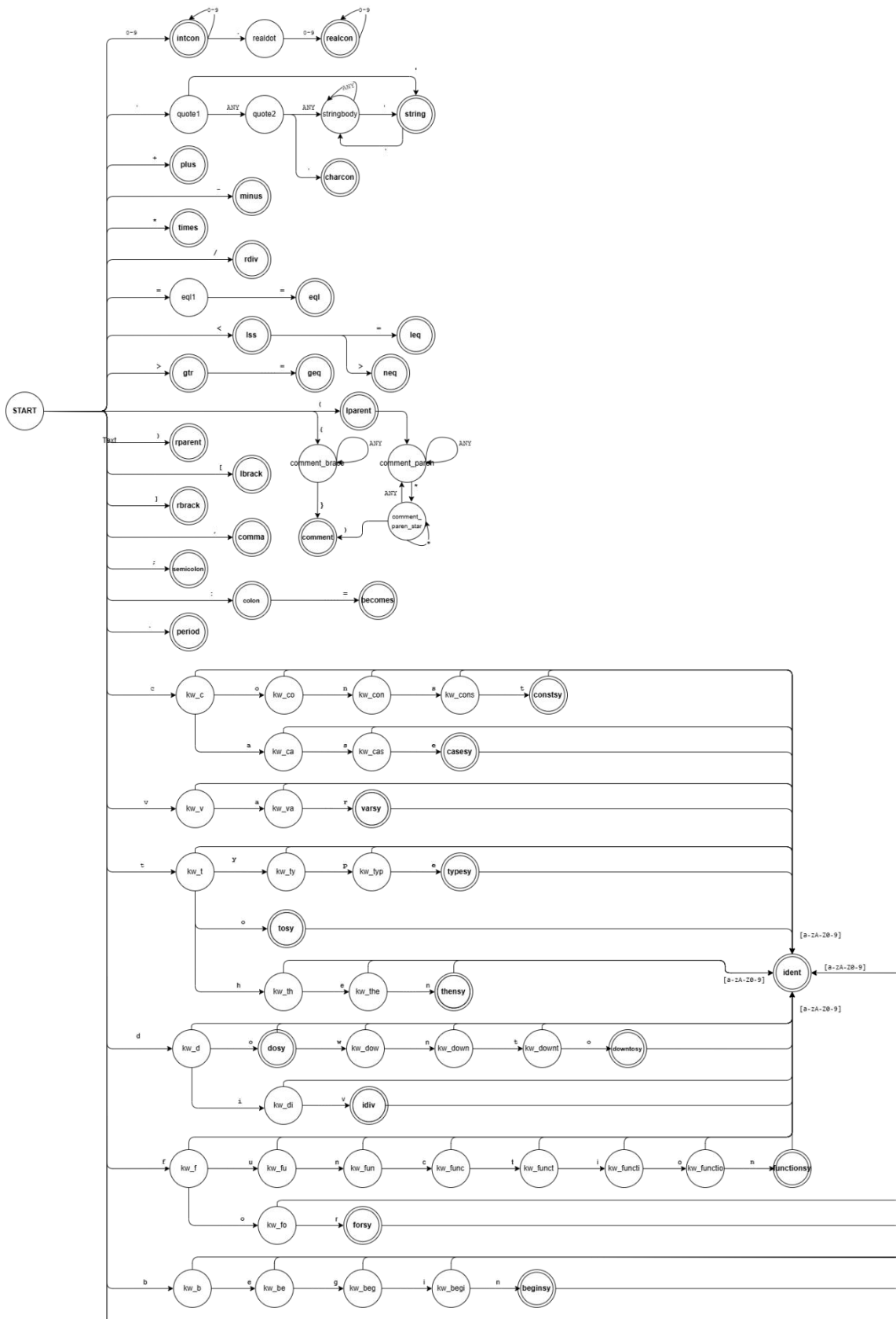
Merupakan fungsi yang menentukan perpindahan state berdasarkan input yang dibaca. Pada diagram, fungsi ini direpresentasikan oleh panah antar state, termasuk transisi spesifik (misalnya +, -) dan transisi umum seperti ANY untuk menangani kelompok karakter tertentu.

- **q_0 (Start State)**

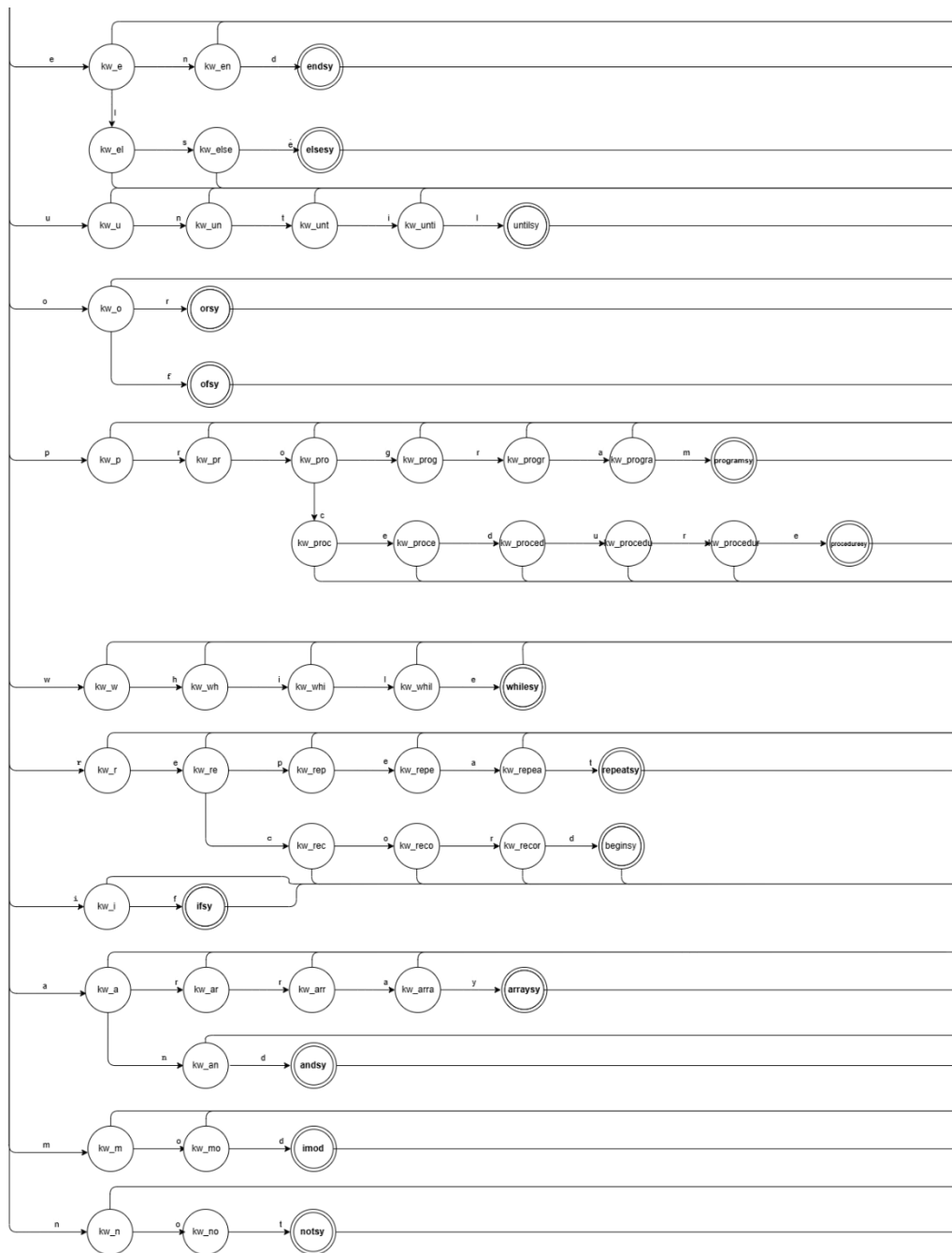
Merupakan state awal tempat proses pembacaan input dimulai, yang ditandai sebagai START pada diagram.

- **F (Final States)**

Merupakan himpunan state akhir yang menandakan bahwa suatu pola input telah dikenali sebagai token yang valid. Pada DFA ini terdapat 52 final state, seperti `intcon`, `realcon`, `ident`, `plus`, `minus`, `lparent`, dan lain-lain, yang masing-masing merepresentasikan jenis token tertentu.



Gambar 2 - Diagram Transisi [1]



Gambar 3 - Diagram Transisi [2]

3.2. Teknologi

Berikut adalah teknologi yang digunakan dalam program:

Tabel 1- Teknologi yang Digunakan

Teknologi yang digunakan	Penjelasan
Bahasa Pemrograman C++ dan STL Library	Digunakan untuk mengimplementasikan lexer, DFA, dan seluruh logika program. Program ini menggunakan library standar C++

	seperti fstream untuk pengolahan file, serta map, set, dan vector untuk pengelolaan data.
Build Tool Makefile	Digunakan untuk mengatur proses kompilasi program secara otomatis.

3.3. Struktur Program

Berikut adalah struktur dari program:

```

MMD-TUBES-IF2224-2026/
|
|—— doc/
|   |—— Laporan-1-MMD. pdf
|
|—— src/
|   |—— DFA. cpp
|   |—— DFA. hpp
|   |—— lexer. cpp
|   |—— lexer. hpp
|   |—— main. cpp
|   |—— dfa_rules. txt
|
|—— test/
|   |—— milestone-1/
|
|—— README. md
|
|—— Makefile

```

3.4. Kelas Utama

Berikut adalah penjelasan dari kelas utama dalam program:

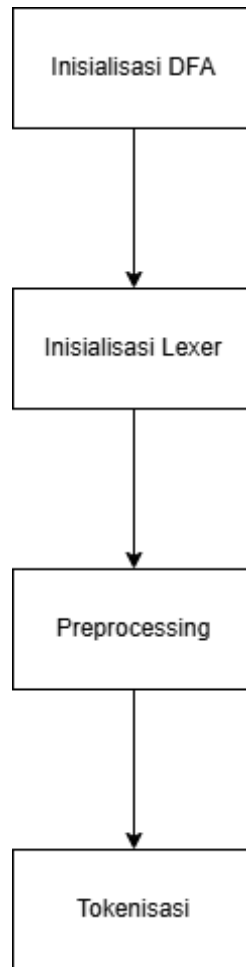
Tabel 2 - Desain Kelas Utama

Nama Kelas	Data/Method	Penjelasan
DFA	string start_state	Menyimpan state awal (start state) dari DFA, yaitu state tempat proses dimulai.
	set<string> final_states	Menyimpan accepting states yang menentukan apakah suatu input diterima.

	<code>string dead_state</code>	Menyimpan dead state, yaitu ketika input tidak sesuai dengan pola yang dikenali.
	<code>transition transitions</code>	Menyimpan fungsi transisi DFA, yaitu pemetaan dari pasangan (state, input) ke state berikutnya.
	<code>map<string, string> token_types</code>	Menyimpan pemetaan antara state akhir dengan jenis token yang dihasilkan.
	<code>set<string> keywords</code>	Menyimpan daftar keyword yang digunakan untuk membedakan identifier biasa dengan keyword dalam bahasa pemrograman.
	<code>DFA(const string& rules_file)</code>	Konstruktor kelas yang digunakan untuk menginisialisasi objek DFA dengan membaca aturan (rules) dari file eksternal. File ini berisi definisi state, transisi, dan informasi lain yang diperlukan.
	<code>transition getTransition()</code>	Method untuk mengembalikan struktur data transisi DFA yang menyimpan hubungan antar state berdasarkan input tertentu.
	<code>bool isFinalState(const string& state) const</code>	Method untuk mengecek apakah suatu state termasuk ke dalam accepting state. Mengembalikan nilai boolean.
	<code>bool isDeadState(const string& state) const</code>	Method untuk mengecek apakah suatu state merupakan dead state.
	<code>void loadRules(const string& filepath)</code>	Method untuk membaca dan memuat aturan DFA dari file, termasuk definisi state, transisi, dan token.
	<code>vector<string> split(const string& str, const string& delim)</code>	Method utilitas untuk memecah string menjadi beberapa bagian berdasarkan delimiter tertentu.
	<code>string toLower(const string& str)</code>	Method utilitas untuk mengubah string menjadi huruf kecil, biasanya digunakan untuk normalisasi input.
	<code>string getDeadState()</code>	Mengembalikan <code>dead_state</code>
	<code>string getStartState()</code>	Mengembalikan <code>start_state</code>
Transition	<code>map<pair<string, char>, string> transitions</code>	Menyimpan fungsi transisi utama DFA, yaitu pemetaan dari pasangan (<i>state saat ini, simbol input</i>) ke state berikutnya. Digunakan untuk

		pencocokan yang bersifat spesifik terhadap suatu karakter.
	map<string, string> wildcard_transitions	Menyimpan transisi khusus yang berlaku untuk semua input tertentu (wildcard). Digunakan ketika tidak ditemukan transisi spesifik, dengan simbol khusus seperti ANY
	void addTransition(string qprev, string input_symbol, string qnext)	Method untuk menambahkan aturan transisi ke dalam struktur data, baik ke transitions maupun wildcard_transitions, berdasarkan simbol input yang diberikan.
	string getNextState(const string& current, const string& dead_state, char inputSymbol)	Method untuk menentukan state berikutnya berdasarkan state saat ini dan simbol input. Menentukan state berikutnya dengan prioritas: transisi spesifik → wildcard → dead state.”
Lexer	char currentChar	Menyimpan karakter yang sedang dibaca dari file sumber.
	int line	Menyimpan nomor baris tempat token ditemukan dalam source code.
	bool EOP	Menandakan apakah sudah mencapai akhir file (End of Program / EOF).
	ifstream scanner	Digunakan untuk membaca file sumber.
	lexer(string filepath)	Konstruktor untuk inisialisasi lexer dengan file input.
	char getCurrent()	Mengembalikan karakter yang sedang diproses.
	bool isEndFile()	Mengecek apakah sudah mencapai akhir file.
	void advance()	Membaca karakter berikutnya dari file, memperbarui currentChar, serta menangani kondisi EOF dan menutup file jika sudah habis.
	void readFile(string filepath)	Membuka file sumber dan langsung membaca karakter pertama dengan memanggil advance().
	void skipWhitespace()	Melewati karakter whitespace seperti spasi, tab, atau newline.
	vector<Token> tokenize(DFA& DFA)	Melakukan proses tokenisasi.

3.5. Alur Kerja Program



Gambar 4 - Alur Kerja Program

Secara umum, program bekerja melalui tiga komponen utama: Lexer, DFA, dan Transition untuk mengubah source code menjadi token. Berikut adalah penjelasan dari setiap langkah-langkahnya:

1. Inisialisasi DFA

Pada tahap ini, program membaca file aturan DFA (`dfa_rules.txt`). Semua aturan transisi akan dimuat ke dalam struktur transition.

2. Inisialisasi Lexer

Pada tahap ini, lexer membuka file source code, kemudian membaca karakter per karakter.

3. Preprocessing

Lexer akan melewati spasi, tab, newline, sekaligus melakukan update nomor baris. Dilakukan supaya DFA hanya memproses karakter yang relevan.

4. Proses Tokenisasi

Merupakan loop utama dari program. Menggunakan fungsi `tokenize()` yang didefinisikan di `lexer.cpp`. Dengan alur sebagai berikut:

- a) Mulai dari
 - `currentState = start_state`
 - `lexeme = ""`
- b) Untuk setiap karakter,
 - Ambil next state:
 - `nextState = getNextState(...)`
- c) Terdapat tiga kemungkinan kasus:
 - CASE 1: Transisi Valid**
 Lanjut membangun token. Ini mencakup transisi spesifik dan wildcard.
`lexeme += currentChar;`
`currentState = nextState;`
 - CASE 2: Masuk Dead State**
 Terjadi apabila tidak ada transisi spesifik dan tidak ada wildcard.
`if (DFA.isFinalState(currentState))`
 - o Jika sebelumnya final:
 - simpan token
 - reset state
 - o Jika tidak:
 - error / stop
- d) EOF Handling
 Jika masih ada lexeme dan state final, maka tetap jadi token.
- e) Output
 Menghasilkan `vector<Token>`.

3.6. Pendekatan Implementasi

Pemilihan implementasi berbasis DFA dengan pendekatan “**character-by-character**” dilakukan untuk memenuhi spesifikasi sekaligus menjaga kesesuaian dengan teori automata. Lexer dirancang membaca input satu karakter setiap iterasi dan menentukan transisi melalui tabel aturan (`dfa_rules.txt`), sehingga seluruh proses tokenisasi bersifat deterministik dan transparan. Pemisahan antara logika pembacaan (lexer) dan definisi transisi (DFA) juga dipilih untuk meningkatkan modularitas, sehingga perubahan aturan bahasa tidak memerlukan perubahan kode utama. Selain itu, penggunaan file aturan eksternal mempermudah debugging karena alur state dapat dilacak langsung dari transisi yang digunakan.

Untuk mengurangi kompleksitas dan meningkatkan maintainability, digunakan pendekatan wildcard seperti ANY dan ANY2 agar transisi yang serupa tidak perlu dituliskan berulang. Implementasi juga mengikuti prinsip *longest match*, di mana lexer terus membaca selama transisi valid sebelum memutuskan token, sehingga menghindari

kesalahan seperti pemecahan token per karakter. Beberapa keputusan desain, seperti membedakan string dan char di tahap akhir (bukan di DFA), dilakukan untuk tetap menjaga sifat deterministik DFA tanpa menambah kompleksitas state. Secara keseluruhan, pendekatan ini menghasilkan implementasi yang efisien, mudah dikembangkan, dan tetap konsisten dengan konsep formal yang dipelajari.

BAB 4

HASIL IMPLEMENTASI

Class transition:

```
1 class transition {
2     private:
3         map<string, string> wildcard_transitions;
4     public:
5         map<pair<string, char>, string> transitions;
6         void addTransition(string qprev, string input_symbol, string qnext);
7         string getNextState(const string& current, const string& dead_state, char inputSymbol);
8     };
```

```
1 string transition::getNextState(const string& currentState, const string& dead_state, char inputSymbol) {
2     if (currentState == dead_state) {
3         return dead_state;
4     }
5
6     auto it = transitions.find({currentState, inputSymbol});
7     if (it != transitions.end()) {
8         return it->second;
9     }
10
11     auto wild_it = wildcard_transitions.find(currentState);
12     if (wild_it != wildcard_transitions.end()) {
13         return wild_it->second;
14     }
15
16     return dead_state;
17 }
18
19 void transition::addTransition(string qprev, string input_symbol, string qnext){
20     if (input_symbol.compare("ANY") == 0) {
21         wildcard_transitions[qprev] = qnext;
22     } else if (input_symbol.compare("ANY2") == 0){
23         for (unsigned char c = 0; c < 128; c++){
24             if ( isalnum(c) && transitions[{qprev,(char)c}] == ""){
25                 transitions[{qprev, (char)c}] = qnext;
26             }
27         }
28     } else {
29         char input_char = input_symbol[0];
30         transitions[{qprev, input_char}] = qnext;
31     }
32 }
```

Class DFA:

```
1  class DFA {
2  public:
3      DFA(const string& rules_file);
4
5      transition getTransition(){ return transitions; };
6      string getDeadState(){ return dead_state; };
7      string getStartState(){ return start_state; };
8      bool isFinalState(const string& state) const;
9      bool isDeadState(const string& state) const;
10
11 private:
12     string start_state;
13     set<string> final_states;
14     string dead_state;
15     transition transitions;
16     map<string, string> token_types;
17     set<string> keywords;
18
19     void loadRules(const string& filepath);
20     vector<string> split(const string& str, const string& delim);
21     string toLower(const string& str);
22 };
```



```

1  DFA::DFA(const string& rules_file) : dead_state("q_trap") {
2      loadRules(rules_file);
3  }
4
5  vector<string> DFA::split(const string& str, const string& cutter) {
6      vector<string> tokens;
7      size_t start = 0, pos;
8
9      while ((pos = str.find(cutter, start)) != string::npos) {
10         string token = str.substr(start, pos - start);
11
12         size_t tstart = token.find_first_not_of(" \t\r\n");
13         size_t tend = token.find_last_not_of(" \t\r\n");
14         if (tstart != string::npos)
15             tokens.push_back(token.substr(tstart, tend - tstart + 1));
16
17         start = pos + cutter.size();
18     }
19
20     string last = str.substr(start);
21     size_t tstart = last.find_first_not_of(" \t\r\n");
22     size_t tend = last.find_last_not_of(" \t\r\n");
23     if (tstart != string::npos)
24         tokens.push_back(last.substr(tstart, tend - tstart + 1));
25
26     return tokens;
27 }
28
29 string DFA::toLowerCase(const string& str) {
30     string lowerStr = str;
31     transform(lowerStr.begin(), lowerStr.end(), lowerStr.begin(), ::tolower);
32     return lowerStr;
33 }
34
35 void DFA::loadRules(const string& filepath) {
36     ifstream file(filepath);
37
38     if (!file.is_open()) {
39         std::cerr << "Error : Gagal membuka file aturan DFA: " << filepath << std::endl;
40         std::cerr << "Pastikan file tersebut ada dan path-nya benar relatif terhadap lokasi eksekusi (root direktori)." << std::endl;
41         exit(EXIT_FAILURE); // Hentikan program secara paksa
42     }
43
44     string line;
45
46     while (getline(file, line)) {
47         if (line.size() <= 1) continue;
48         if (line.find("#") == 0) continue;
49         if (line.find("startstate") == 0) {
50             auto parts = split(line, "|");
51             start_state = parts[1];
52         } else if (line.find("deadstate") == 0) {
53             auto parts = split(line, "|");
54             dead_state = parts[1];
55         } else if (line.find("finalstate") == 0) {
56             auto parts = split(line, "|");
57             string states_str = parts[1];
58             auto states = split(states_str, ",");
59             final_states.insert(states.begin(), states.end());
60         } else {
61             auto parts = split(line, " ");
62             if (parts.size() == 2) {
63                 parts.push_back(" ");
64             }
65             transitions.addTransition(parts[0], parts[1], parts[2]);
66         }
67     }
68 }
69
70 bool DFA::isFinalState(const string& state) const {
71     return final_states.find(state) != final_states.end();
72 }
73
74 bool DFA::isDeadState(const string& state) const {
75     return state == dead_state;
76 }

```

Class Lexer:

```
1  class Lexer {
2      private:
3          char currentChar;
4          int line;
5          bool EOP;
6          ifstream scanner;
7      public:
8          Lexer();
9          char getCurrent(){ return currentChar; };
10         bool isEndFile(){ return EOP; };
11         void advance();
12         void readFile(string filepath);
13         void skipWhitespace();
14         bool generateToken(string srcFile, string destFile);
15     };
```

```
1  void Lexer::advance(){
2      char c = scanner.get();
3      EOP = (currentChar == EOF);
4      if(c == EOF){
5          cout << endl << "File mencapai akhir (EOF)" << endl;
6          EOP = true;
7          currentChar = EOF;
8          scanner.close();
9          return;
10     }
11
12     currentChar = c;
13 }
14
15 void Lexer::readFile(string filepath){
16     scanner.open(filepath);
17     if(!scanner){
18         cout << "Gagal membuka file pada path " << filepath << endl;
19         EOP = true;
20         currentChar = EOF;
21         scanner.close();
22         return;
23     }
24     line = 0;
25     advance();
26 }
27
28 void Lexer::skipWhitespace(){
29     while(isspace(currentChar)){
30         if(currentChar == '\n') line++;
31         advance();
32     }
33 }
```

BAB 5

EKSPERIMEN

❖ Kasus uji 1

Input	Output
<pre>src > input.txt 1 program Hello; 2 3 var 4 =, b: integer; 5 6 begin 7 a := 5; 8 b := a + 10; 9 writeln('Result = ', b); 10 end.</pre>	<pre>src > output.txt 1 programsy 2 ident (Hello) 3 semicolon 4 varsy 5 eql 6 comma 7 ident (b) 8 colon 9 ident (integer) 10 semicolon 11 beginsy 12 ident (a) 13 becomes 14 intcon (5) 15 semicolon 16 ident (b) 17 becomes 18 ident (a) 19 plus 20 intcon (10) 21 semicolon 22 ident (writeln) 23 lparent 24 string ('Result = ') 25 comma 26 ident (b) 27 rparent 28 semicolon 29 endsy 30 period</pre>

❖ Kasus uji 2

Input	Output
<pre>src > input.txt 1 program LuasLingkaran 2 const PI = 3.14; 3 procedure PrintLuas(r: integer); 4 var luas: real; 5 begin 6 luas = PI * r * r; 7 end;</pre>	<pre>1 programsy 2 ident (LuasLingkaran) 3 semicolon 4 constsy 5 ident (PI) 6 unknown (=) 7 realcon (3.14) 8 semicolon 9 proceduresy 10 ident (PrintLuas) 11 lparent 12 ident (r) 13 colon 14 ident (integer) 15 rparent 16 semicolon 17 varsy 18 ident (luas) 19 colon 20 ident (real) 21 semicolon 22 beginsy 23 ident (luas) 24 unknown (=) 25 ident (PI) 26 times 27 ident (r) 28 times 29 ident (r) 30 semicolon 31 endsy 32 semicolon</pre>

❖ Kasus uji 3

Input	Output
<pre>{ Ini Komen } program TesKomentar; var (* Ini komen juga Ini komen baris dua *) angka: integer; begin angka = 100; { Ini komen lagi } (* lorem ipsum writeln('Ini diabaikan karena dalam komentar'); *) end.</pre>	<pre>1 comment ({ Ini Komen }) 2 programsy 3 ident (TesKomentar) 4 semicolon 5 varsy 6 comment ((* Ini komen juga 7 Ini komen baris dua *)) 8 ident (angka) 9 colon 10 ident (integer) 11 semicolon 12 beginsy 13 ident (angka) 14 unknown (=) 15 intcon (100) 16 semicolon 17 comment ({ Ini komen lagi }) 18 comment ((* lorem ipsum 19 writeln('Ini diabaikan karena dalam komentar'); 20 *)) 21 endsy 22 period</pre>

❖ Kasus uji 4

Input	Output
<pre>1 program skibidi; 2 3 var x, y: integer; 4 begin 5 x = 15; 6 repeat 7 y = x mod 2; 8 x = x div 2; 9 until x <= 0; 10 end. 11</pre>	<pre>1 programsy 2 ident (skibidi) 3 semicolon 4 varsy 5 ident (x) 6 comma 7 ident (y) 8 colon 9 ident (integer) 10 semicolon 11 beginsy 12 ident (x) 13 unknown (=) 14 intcon (15) 15 semicolon 16 repeatsy 17 ident (y) 18 unknown (=) 19 ident (x) 20 imod 21 intcon (2) 22 semicolon 23 ident (x) 24 unknown (=) 25 ident (x) 26 idiv 27 intcon (2) 28 semicolon 29 untilsy 30 ident (x) 31 leq 32 intcon (0) 33 semicolon 34 endsy 35 period</pre>

❖ Kasus uji 5

Input	Output
<pre> 1 program tesDownTo; 2 3 var i: integer; 4 begin 5 for i = 10 downto 1 do 6 begin 7 case i of 8 1: writeln('Satu'); 9 end; 10 end; 11 end. </pre>	<pre> 1 programsy 2 ident (tesDownTo) 3 semicolon 4 varsy 5 ident (i) 6 colon 7 ident (integer) 8 semicolon 9 beginsy 10 forsy 11 ident (i) 12 unknown (=) 13 intcon (10) 14 downtosy 15 intcon (1) 16 dosy 17 beginsy 18 casesy 19 ident (i) 20 ofsy 21 intcon (1) 22 colon 23 ident (writeln) 24 lparent 25 string ('Satu') 26 rparent 27 semicolon 28 endsy 29 semicolon 30 endsy 31 semicolon 32 endsy 33 period </pre>

BAB 6

KESIMPULAN DAN SARAN

6.1. Kesimpulan

Pada milestone ini telah berhasil diimplementasikan sebuah lexer untuk bahasa Arion yang bekerja berdasarkan konsep Deterministic Finite Automata (DFA). Program mampu membaca source code secara karakter demi karakter dan mengelompokkan input menjadi token-token yang valid seperti keyword, identifier, konstanta, operator, delimiter, serta komentar. DFA yang digunakan terdiri dari banyak state dan transisi yang merepresentasikan seluruh pola token, sehingga proses lexical analysis dapat dilakukan secara sistematis, deterministik, dan sesuai dengan teori automata yang dipelajari. Selain itu, penggunaan file aturan eksternal (dfa_rules.txt) membuat sistem lebih fleksibel dan mudah dikembangkan.

Secara keseluruhan, implementasi ini telah memenuhi spesifikasi yang diberikan, baik dari segi fungsionalitas maupun pendekatan teoritis. Program mampu menangani berbagai kasus uji dengan baik, termasuk kombinasi token yang kompleks, serta memiliki struktur yang modular sehingga memudahkan pengembangan lebih lanjut. Pendekatan yang digunakan juga berhasil menjaga keseimbangan antara ketepatan formal DFA dan efisiensi implementasi dalam kode.

6.2. Saran

Untuk pengembangan selanjutnya, program dapat ditingkatkan dengan menambahkan error handling yang lebih informatif, seperti menampilkan jenis kesalahan dan posisi detail pada source code. Hal ini akan sangat membantu dalam proses debugging dan meningkatkan usability dari interpreter. Selain itu, pengelolaan DFA dapat lebih dioptimalkan, misalnya dengan melakukan minimisasi state atau penyederhanaan transisi agar lebih efisien dan mudah dipahami.

Selanjutnya, sistem ini dapat dikembangkan ke tahap berikutnya dalam proses compiler/interpreter, yaitu syntax analysis (parser) dan semantic analysis, sehingga dapat membentuk pipeline interpretasi yang lengkap. Integrasi dengan visualisasi DFA atau logging transisi juga dapat dipertimbangkan untuk membantu proses pembelajaran dan debugging. Dengan pengembangan tersebut, program tidak hanya berfungsi sebagai lexer, tetapi dapat menjadi fondasi yang kuat untuk membangun interpreter bahasa Arion secara utuh.

LAMPIRAN

Repository GitHub: <https://github.com/SyaqinaOctavia/MMD-Tubes-IF2224-2026.git>

Diagram Transisi DFA:

https://drive.google.com/file/d/1nEhBtFxIlxVPvB3II4ZGqLZyLrkrqoNg/view?usp=drive_link

Tabel 3 - Pembagian Tugas

NIM	Kontribusi Tugas	Persentase
13524040	Tokenizer, Testing program, Testcase input-output, diagram DFA	25%
13524042	Penulisan laporan, DFA rules, diagram DFA	25%
13524048	Class lexer, class DFA, diagram DFA	25%
13524088	Penulisan laporan, DFA rules, Diagram DFA	25%

REFERENSI

John E., Rajeev M., dan Jeffrey D. *Introduction to Automata Theory, Languages, and Computation 3rd Edition*

https://cdn-edunex.itb.ac.id/29161-Formal-Language-Theory-and-Automata/1629640939613_Introduction-to-Automata-Theory,-Languages,-and-Computation-Edition-3.pdf

GeeksForGeeks. *Introduction to Interpreter*. Diakses 1 April 2026

<https://www.geeksforgeeks.org/compiler-design/introduction-to-interpreters/>

GeeksForGeeks. *Introduction to Lexical Analysis*. Diakses 1 April 2026

<https://www.geeksforgeeks.org/compiler-design/introduction-of-lexical-analysis/>